Wine

Homework #5

Naomi Buell Richie Rivera Alexander Simon Nana Frimpong
Said Naqwe

2026-05-16

1 Introduction

In this homework assignment, we explore, analyze, and model a data set containing information on approximately 12,000 commercially available wines. Our objective is to build a count regression model to predict the number of cases of wine that will be sold given certain properties of the wine.

2 Data Exploration

```
# Load the training and evaluation datasets
wine_train <- read_csv("./data/wine-training-data.csv")
wine_eval <- read_csv("./data/wine-evaluation-data.csv")

# Count rows and columns
nrows <- wine_train |> nrow()
ncols <- wine_train |> length()

# Get missingness
perc_complete <- round(wine_train |> n_complete() / nrows / ncols * 100, 2)
```

The wine training data set contains 12795 rows and 16 features, and is 95.99 percent complete. We will need to address the missingness in the data preparation section. All variables are numeric; most are continuous variables describing chemical characteristics of the wines, while others like `STARS` and `LabelAppeal` are discrete. The target variable is `TARGET`, which represents the number of cases purchased.

Table 1: Summary statistics for all variables in the training dataset

Variable	Completeness rate	Mean	Min	P25	P50	P75	Max
INDEX	1.00	8100.0000	1.00	4000.000	8.1e+03	1.2e+04	16000.0
TARGET	1.00	3.0000	0.00	2.000	3.0e+00	4.0e+00	8.0
FixedAcidity	1.00	7.1000	-18.00	5.200	6.9e+00	9.5e+00	34.0
VolatileAcidity	1.00	0.3200	-2.80	0.130	2.8e-01	6.4e-01	3.7
CitricAcid	1.00	0.3100	-3.20	0.030	3.1e-01	5.8e-01	3.9
ResidualSugar	0.95	5.4000	-130.00	-2.000	3.9e+00	1.6e+01	140.0
Chlorides	0.95	0.0550	-1.20	-0.031	4.6e-02	1.5e-01	1.4
FreeSulfurDioxide	0.95	31.0000	-560.00	0.000	3.0e+01	7.0e+01	620.0
TotalSulfurDioxide	0.95	120.0000	-820.00	27.000	1.2e+02	2.1e+02	1100.0
Density	1.00	0.9900	0.89	0.990	9.9e-01	1.0e+00	1.1
pH	0.97	3.2000	0.48	3.000	3.2e+00	3.5e+00	6.1
Sulphates	0.91	0.5300	-3.10	0.280	5.0e-01	8.6e-01	4.2
Alcohol	0.95	10.0000	-4.70	9.000	1.0e+01	1.2e+01	26.0
LabelAppeal	1.00	-0.0091	-2.00	-1.000	0.0e+00	1.0e+00	2.0
AcidIndex	1.00	7.8000	4.00	7.000	8.0e+00	8.0e+00	17.0
STARS	0.74	2.0000	1.00	1.000	2.0e+00	3.0e+00	4.0

Table 1 presents descriptive characteristics of the training data.

```
# Get correlation matrix
wine_train_cor <- wine_train |>
    select(where(is.numeric)) |>
    cor(use = "pairwise.complete.obs")

wine_train_cor_plot <- wine_train_cor
wine_train_cor_plot[abs(wine_train_cor_plot) < 0.05] <- NA

corrplot(wine_train_cor_plot, method = "circle", type = "upper", sig.level = 0.05, insig = "bla

most_corr_w_target_flag <- wine_train_cor |>
    as.data.frame() |>
    rownames_to_column(var = "Variable") |>
    filter(Variable != "TARGET") |>
    transmute(
```

```
        Variable,
        Correlation = TARGET,
        Abs_Correlation = abs(TARGET)
    ) |>
    arrange(desc(Abs_Correlation)) |>
    filter(Abs_Correlation > 0.05) |>
    pull(Variable)
```

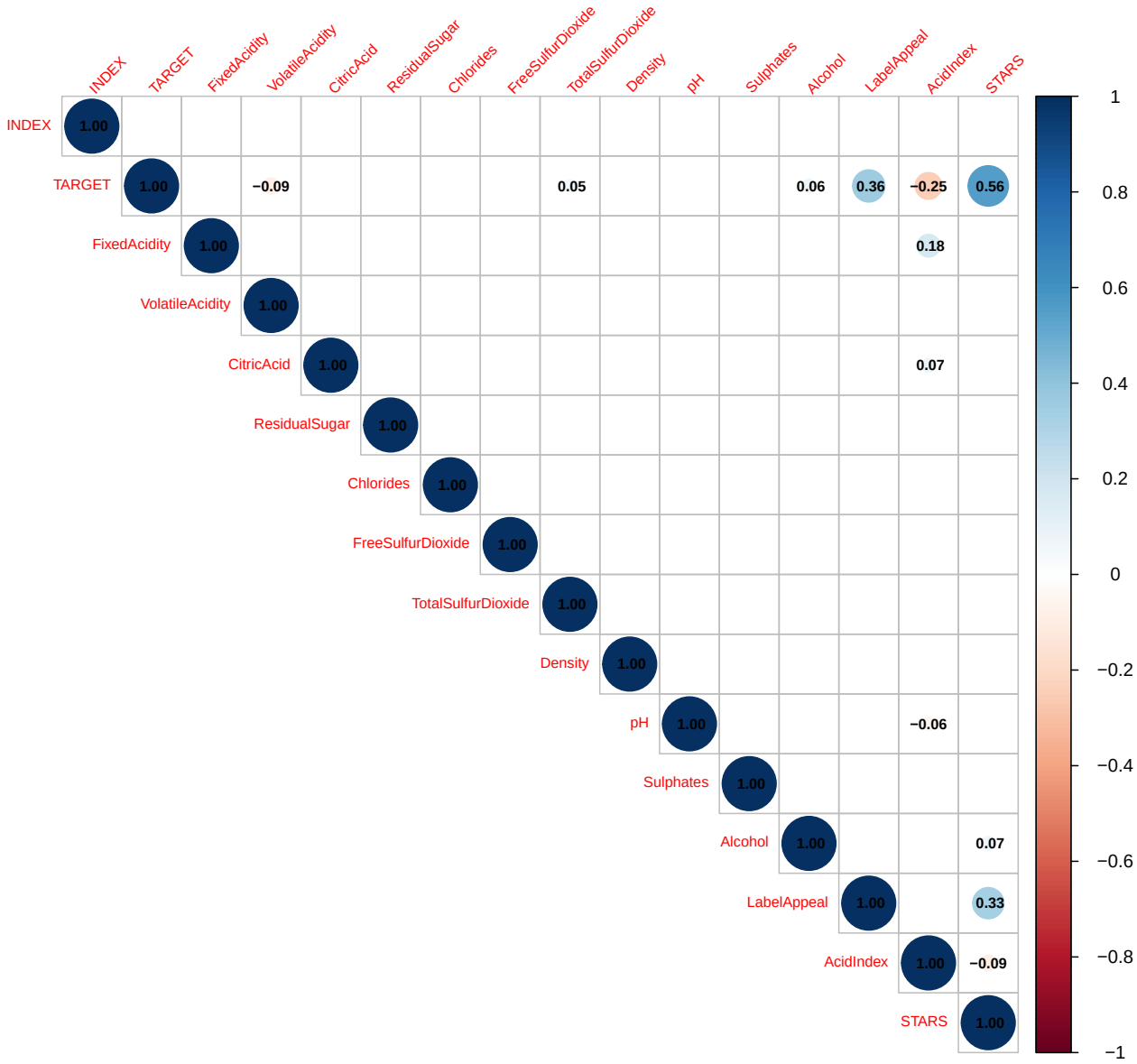

Figure 1: Correlation matrix of all variables in the training dataset. Note: Correlations with absolute value less than 0.05 are suppressed.

The correlation between all variables are shown Figure 1. The variables most correlated with

`TARGET`, the number of cases purchased, are STARS, LabelAppeal, AcidIndex, VolatileAcidity, Alcohol, TotalSulfurDioxide. This means that wines with higher ratings (`STARS`), more appealing labels (`LabelAppeal`), lower acidity (`AcidIndex` and `VolatileAcidity`), higher alcohol content (`Alcohol`), and higher sulfur dioxide levels (`TotalSulfurDioxide`) tend to have higher sales. These variables will be important to consider in our modeling process, as they may have strong predictive power for the target variable.

It is important to note that label appeal and star ratings are likely to be highly correlated with each other, which may lead to multicollinearity issues in a regression model. Similarly, `AcidIndex` and `VolatileAcidity` are likely to be correlated, which may also lead to multicollinearity issues. We will need to consider this when selecting variables for our model.

Lastly, we also looked at the distribution of each variable to check for skewness and outliers.

```
# Keep only continuous
continuous_vars <- wine_train |>
    select(where(is.numeric), -INDEX) |>
    names()

wine_train |>
    select(all_of(continuous_vars)) |>
    pivot_longer(cols = everything(), names_to = "variable", values_to = "value") |>
    ggplot(aes(x = value)) +
    geom_histogram(bins = 30, fill = "steelblue", color = "white") +
    facet_wrap(~ variable, scales = "free", ncol = 4) +
    theme_minimal() +
    theme(panel.grid = element_blank()) +
    labs(x = NULL, y = "Count")
```

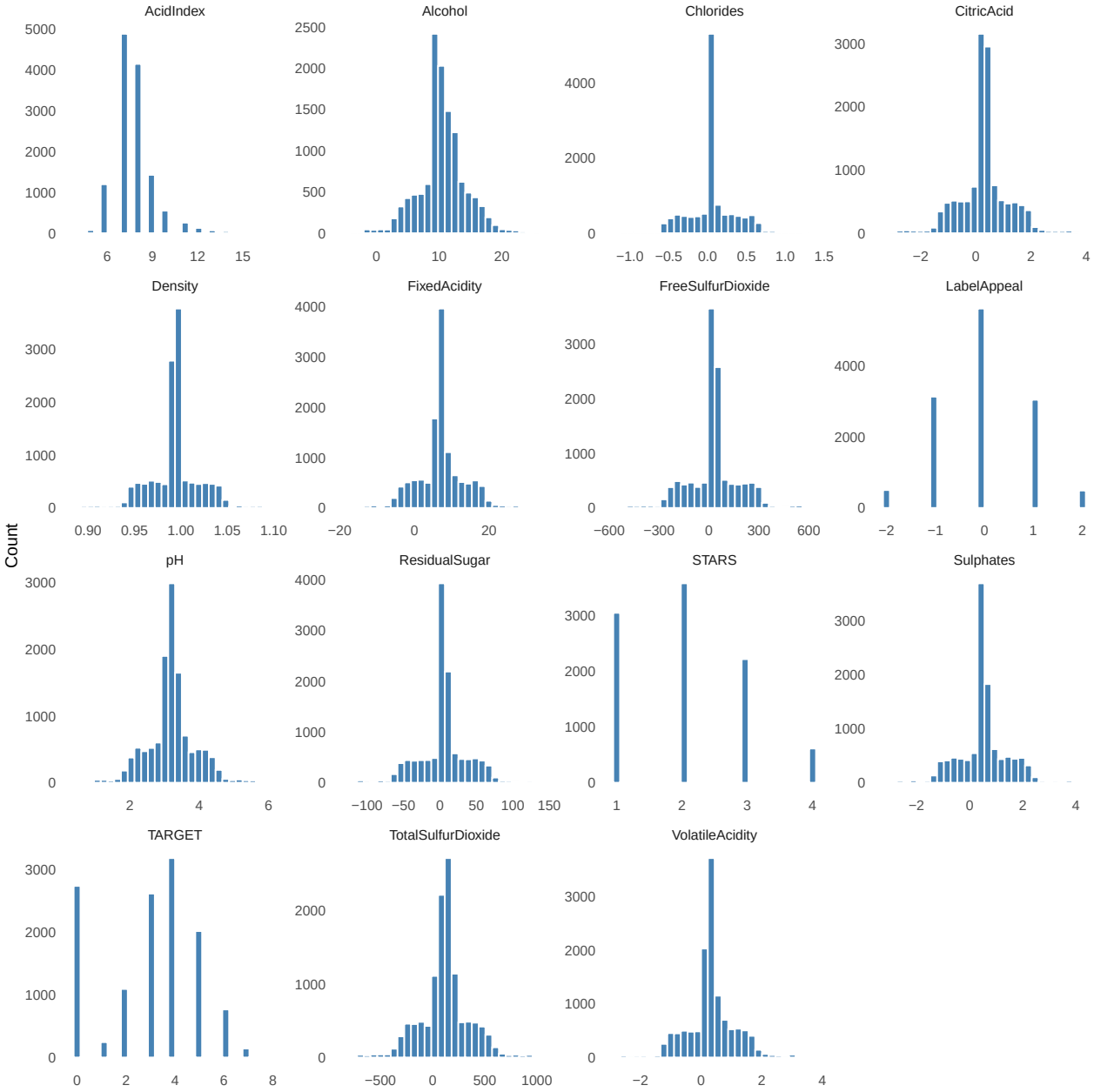

Figure 2: Distribution of continuous variables in the training dataset.

`AcidIndex` is right-skewed, with fewer lines having high acidity. Many other variables appear normally distributed, such as `Alcohol`, `pH`, and `TotalSulfurDioxide`. `STARS` is a discrete variable with values from 1 to 4, and is right-skewed with more wines having lower ratings. `LabelAppeal` is also discrete, with values from -2 to 2, and is normally distributed around 0. There do not appear to be any extreme outliers in the data, although many variables have high frequency of median values, which may indicate that many wines have similar characteristics in these dimensions. Note that `TARGET` has a high frequency of 0 values, which may indicate that many wines have no sales. This may require special consideration in our modeling process.

3 Data Preparation

For the wine data used in this assignment, the first step in data preparation was to examine whether any variables contained missing values. Unlike the crime dataset from Homework 3, the wine dataset contains missing values across several predictor variables in both the training and evaluation datasets. Because of this, appropriate missing-value treatment was required before proceeding to modeling.

To address missing data, median imputation was applied to all numeric predictor variables. Median imputation was selected because many of the chemical variables (such as sulfur dioxide levels, residual sugar, and alcohol content) are skewed and may contain outliers. Using the median provides a robust estimate that is less sensitive to extreme values than the mean.

In addition to imputation, missingness indicator variables were created for each predictor. This was done because the assignment specifically notes that missing values themselves may be predictive of the response variable. By creating binary flags indicating whether a value was originally missing, we preserve this potential signal for use in the modeling stage.

Next, several transformations were applied to improve the distribution of the predictors. Variables such as `alcohol`, `residualsugar`, `freesulfurdioxide`, and `totalsulfurdioxide` exhibited right-skewed distributions, so log transformations were created using `log1p()`. These transformations reduce skewness and help stabilize relationships between predictors and the count response variable.

We also created a set of engineered variables to capture relationships between predictors that may not be fully represented by the original variables. The variable `sulfur_ratio` measures the relationship between total and free sulfur dioxide, while `acidity_ratio` compares volatile acidity to fixed acidity. The variable `density_alcohol_ratio` captures the interaction between density and alcohol content. Finally, `quality_score` combines expert rating (`stars`) and marketing appeal (`labelappeal`) to reflect an overall perceived quality of the wine.

In addition, the `stars` variable was transformed into categorical buckets (low, medium, high) to capture potential non-linear effects. These buckets were then converted into dummy variables for use in regression models.

Finally, the cleaned training and evaluation datasets were stored as `wine_train_prep` and `wine_eval_prep` so they can be used directly in the modeling section.

```
library(tidyverse)
library(janitor)
library(fastDummies)

# Remove INDEX variable and clean column names
wine_train_prep <- wine_train |>
  clean_names() |>
```

```
  select(-any_of("index"))

wine_eval_prep <- wine_eval |>
  clean_names() |>
  select(-any_of("index"))

# Identify numeric predictors
numeric_vars <- wine_train_prep |>
  select(where(is.numeric)) |>
  select(-target) |>
  names()

# Create missingness flags
wine_train_prep <- wine_train_prep |>
  mutate(across(
    all_of(numeric_vars),
    ~ ifelse(is.na(.), 1, 0),
    .names = "{.col}_missing"
  ))

wine_eval_prep <- wine_eval_prep |>
  mutate(across(
    all_of(numeric_vars),
    ~ ifelse(is.na(.), 1, 0),
    .names = "{.col}_missing"
  ))

# Median imputation
for (v in numeric_vars) {
  med_val <- median(wine_train_prep[[v]], na.rm = TRUE)

  wine_train_prep[[v]][is.na(wine_train_prep[[v]])] <- med_val
  wine_eval_prep[[v]][is.na(wine_eval_prep[[v]])] <- med_val
}

# Transformations and feature engineering
wine_train_prep <- wine_train_prep |>
  mutate(
    log_alcohol = log1p(alcohol),
```

```
    log_residual_sugar = log1p(residual_sugar),
    log_free_sulfur_dioxide = log1p(free_sulfur_dioxide),
    log_total_sulfur_dioxide = log1p(total_sulfur_dioxide),

    sulfur_ratio = total_sulfur_dioxide / (free_sulfur_dioxide + 1),
    acidity_ratio = volatile_acidity / (fixed_acidity + 1),
    density_alcohol_ratio = density / (alcohol + 1),
    quality_score = stars * label_appeal,

    stars_bucket = case_when(
      stars <= 2 ~ "low",
      stars == 3 ~ "medium",
      stars >= 4 ~ "high",
      TRUE ~ "unknown"
    )
  )
```

```
Warning in log1p(alcohol): NaNs produced
```

```
Warning in log1p(residual_sugar): NaNs produced
```

```
Warning in log1p(free_sulfur_dioxide): NaNs produced
```

```
Warning in log1p(total_sulfur_dioxide): NaNs produced
```

```
wine_eval_prep <- wine_eval_prep |>
  mutate(
    log_alcohol = log1p(alcohol),
    log_residual_sugar = log1p(residual_sugar),
    log_free_sulfur_dioxide = log1p(free_sulfur_dioxide),
    log_total_sulfur_dioxide = log1p(total_sulfur_dioxide),

    sulfur_ratio = total_sulfur_dioxide / (free_sulfur_dioxide + 1),
    acidity_ratio = volatile_acidity / (fixed_acidity + 1),
    density_alcohol_ratio = density / (alcohol + 1),
    quality_score = stars * label_appeal,

    stars_bucket = case_when(
      stars <= 2 ~ "low",
      stars == 3 ~ "medium",
      stars >= 4 ~ "high",
      TRUE ~ "unknown"
```

```
    )
  )
```

```
Warning in log1p(alcohol): NaNs produced
```

```
Warning in log1p(residual_sugar): NaNs produced
```

```
Warning in log1p(free_sulfur_dioxide): NaNs produced
```

```
Warning in log1p(total_sulfur_dioxide): NaNs produced
```

```
# Convert categorical bucket to dummy variables
wine_train_prep <- fastDummies::dummy_cols(
  wine_train_prep,
  select_columns = "stars_bucket",
  remove_selected_columns = TRUE,
  remove_first_dummy = TRUE
)

wine_eval_prep <- fastDummies::dummy_cols(
  wine_eval_prep,
  select_columns = "stars_bucket",
  remove_selected_columns = TRUE,
  remove_first_dummy = TRUE
)

# Check for remaining missing values in training data
wine_train_prep |>
  summarise(across(everything(), ~ sum(is.na(.)))) |>
  pivot_longer(everything(), names_to = "Variable", values_to = "Missing") |>
  filter(Missing > 0)
```

```
# A tidytable: 4 x 2
  Variable                Missing
  <chr>                     <int>
1 log_alcohol                  81
2 log_residual_sugar         3103
3 log_free_sulfur_dioxide    3026
4 log_total_sulfur_dioxide   2500
```

```
# Check for remaining missing values in evaluation data
wine_eval_prep |>
  summarise(across(everything(), ~ sum(is.na(.)))) |>
```

```
  pivot_longer(everything(), names_to = "Variable", values_to = "Missing") |>
  filter(Missing > 0)
```

```
# A tidytable: 5 x 2
  Variable                Missing
  <chr>                     <int>
1 target                     3335
2 log_alcohol                  19
3 log_residual_sugar          814
4 log_free_sulfur_dioxide     771
5 log_total_sulfur_dioxide    634
```

3.0.1 Linearity Assumption Summary

The results above indicate that the assumption of linearity is satisfied for all independent variables. However, several were flagged as being unlikely to have predictive value in a multivariate linear model.

3.1 Data Splits

First, we will split our dataset into a training and testing set. We will use the training set to build our models and the testing set to evaluate their performance. The training set will be used to fit the models, while the testing set will be used to assess how well the models generalize to new data.

```
set.seed(19940211)
train_indices <- sample(nrow(crime_clean), size = 0.8 * nrow(crime_clean))
crime_train_split <- crime_clean[train_indices, ]
crime_test_split <- crime_clean[-train_indices, ]
```

4 Check assumptions for binary logistic regression

1. Binary Outcome

```
# Confirming that the target is binary
crime_clean |>
    count(target) |>
    mutate(prop = n / sum(n))
```

The response variable “target” is confirmed to be a binary, therefore it takes two values (0: crime rate below the median) and (1: crime rate above the median). In the 466 neighborhoods in training data set 237 (50.9%) fall in the class of low crime and 228 (49.1)% are in the high crime neighborhood.

There is no class imbalance, due to the near equal split meaning there is unlikely to be bias towards one class.

2. Independence of Observations

Each record in the dataset represents a distinct neighborhood, and there is no repeated-measures or clustered structure in the data. Therefore, the independence of observations assumption is satisfied by design and requires no further adjustment.

3. No Multicollinearity (VIF Check)

```
library(car)

# Fit a reference logistic model with original predictors
model_ref <- glm(
    target ~ zn + indus + chas + nox + rm + age + dis + rad + tax + ptratio
        + lstat + medv,
    data = crime_clean, family = binomial
)

# Compute VIF
vif_vals <- vif(model_ref) |>
    as.data.frame() |>
    rownames_to_column("Variable") |>
    rename(VIF = 2) |>
    arrange(desc(VIF))

vif_vals |> knitr::kable(caption = "Variance Inflation Factors for Reference Model")
```

The VIF values is calculated for all predictors to check for multicollinearity. A VIF above 10 is considered to be severe, while between 5 and 10 should warrant caution. None of the variable exceed a threshold of 10, indicating multicollinearity is not present.The highest VIF belongs to medv (median home value, VIF = 8.12), followed by rm (average rooms per dwelling, VIF = 5.81). These two variables are moderate and do not need requrie any removal. The remaining predictors fall below 5 and are acceptable levels of multicollinearity. There was strong pairwise correlation between “rad” and “tax” in the exploration section, howevr it is confirmed that multicollinearity does not exist in the dataset.

4. Linearity of Log-Odds (Box-Tidwell Test)

```
# Box-Tidwell test: add interaction of each continuous predictor with its log
# to test whether the log-odds relationship is linear
continuous_vars <- c(
```

```
    "zn", "indus", "nox", "rm", "age", "dis", "rad", "tax",
    "ptratio", "lstat", "medv"
)

# Create interaction terms x * log(x) for each continuous predictor
# (use log1p to handle zeros)
crime_bt <- crime_clean |>
    mutate(across(all_of(continuous_vars), ~ . * log1p(.), .names = "{.col}_logx"))

bt_formula <- as.formula(paste(
    "target ~",
    paste(continuous_vars, collapse = " + "),
    "+",
    paste(paste0(continuous_vars, "_logx"), collapse = " + ")
))

model_bt <- glm(bt_formula, data = crime_bt, family = binomial)

# Extract p-values for the interaction terms only
bt_results <- summary(model_bt)$coefficients |>
    as.data.frame() |>
    rownames_to_column("Term") |>
    filter(str_detect(Term, "_logx")) |>
    mutate(
        Variable = str_remove(Term, "_logx"),
        Significant = `Pr(>|z|)` < 0.05
    ) |>
    select(Variable, Estimate, `Pr(>|z|)`, Significant) |>
    mutate(across(where(is.numeric), ~ signif(., 3)))

bt_results |> knitr::kable(caption = "Box-Tidwell Test for Linearity of Log-Odds")
```

The Box-Tidwell test was used to assess whether each continuous predictor has a linear relationship with the log-odds of the outcome. A p-value greater than 0.05 violate the assumption. There are three variables that were flagged as significant:rm ($p = 0.003$), dis ($p < 0.001$), and tax ($p < 0.001$). This indicates that these predictors do not have strict lienear relationship with log-odds of high crime. Log transformation were applied to these variables in the data preparation section in order to be used in the model building. The remaining eight predictors did not have any evidence of violating the linearity assumption.

5 Build models

We compared three models. Our first model included all the variables in the cleaned training dataset. The second model aimed to reduce multicollinearity by omitting variables that had correlation coefficients $|r| > 0.7$ (see Data Exploration section). Finally, we generated the third model using backward stepwise selection from a full model that included transformed variables and newly created variables (see Data Preparation section).

5.1 Model 1. All untransformed variables

As shown below, the most significant predictors were `nox` and `rad` ($P < 0.001$). In addition, `age` and `ptratio` were significant at the $P < 0.01$ level, and `dis` and `tax` were significant at the $P < 0.05$ level. The AIC for this model is `r\ AIC(model1)`.

```
model1 <- glm(
    formula = target ~ zn + indus + chas + nox + rm + age + dis + rad + tax +
        ptratio + lstat + medv,
    family = "binomial",
    data = crime_train_split
)

summary(model1)
```

5.2 Model 2. Reduce multicollinearity

In the Data Exploration section, we showed that `rad` and `tax` were strongly correlated (r=0.91), which suggests that only one of these variables is needed in Model 1 (we chose to retain `rad` as it was more highly significant). Similarly, `medv` was shown to be potentially multicollinear with `lstat` (r=0.74) and `rm` (r=0.71), so we removed the latter variables.

In addition, the correlation between `nox` (predictor) and `target` (response) suggests that `nox` is an important predictor, so we removed its potential collinear pairs `age` and `dis`. Although `nox` was moderately correlated with `indus` (r=0.76), we retained `indus` because removal of both `indus` and `tax` (highly correlated with `rad`) would eliminate a potentially informative predictor.

These changes resulted in a model with AIC `r\ AIC(model2)`.

```
model2 <- glm(
    formula = target ~ zn + indus + chas + nox + rad + ptratio + medv,
    family = "binomial",
    data = crime_train_split
)
```

```
summary(model2)
```

5.3 Model 3. Backward stepwise variable selection

For this model, we performed backward stepwise selection from the set of variables including transformed and newly created variables.

The best model had `nox`, `tax_rad_ratio`, `ptratio`, and `age` as the most significant predictors (P<0.001). In addition, `log_dis` and `rm_lstat_ratio` were significant at the P<0.05 level. The AIC of this model was `r\ AIC(model3)`, which was lower than that of Models 1 and 2.

```
# Full model includes all variables
model_full <- glm(
    formula = target ~ log_zn + indus + chas + nox + rm + age +
        log_dis + log_rad + log_tax + ptratio + log_lstat + medv +
        nox_indus + rm_lstat_ratio + tax_rad_ratio,
    family = "binomial",
    data = crime_train_split
)

# Select variables
model3 <- step(model_full, direction = "backward")

summary(model3)
```

Of these three models, model 3 has the lowest AIC. In the next section, we will evaluate the performance of these models to more determine which one is best for predicting the `target` variable.

6 Select Models

```
# Creating a custom helper function to extract all the required metrics
# explicitly evaluating on the provided training dataset.
get_logistic_metrics <- function(model, model_name, data, target_var = "target") {
    # 1. Get predicted probabilities and classes
    probs <- predict(model, newdata = data, type = "response")
    preds <- factor(ifelse(probs > 0.5, 1, 0), levels = c(0, 1))
    actuals <- factor(data[[target_var]], levels = c(0, 1))

    # 2. Generate Confusion Matrix and derived metrics
    cm <- confusionMatrix(preds, actuals, positive = "1")
```

```
    # 3. Calculate AUC
    roc_obj <- roc(data[[target_var]], probs, quiet = TRUE)
    auc_val <- as.numeric(pROC::auc(roc_obj))

    # 4. Return results as a tibble
    tibble(
        Model = model_name,
        Deviance = model$deviance,
        Accuracy = cm$overall["Accuracy"],
        Error_Rate = 1 - cm$overall["Accuracy"],
        Precision = cm$byClass["Precision"],
        Sensitivity = cm$byClass["Sensitivity"], # Also Recall
        Specificity = cm$byClass["Specificity"],
        F1 = cm$byClass["F1"],
        AUC = auc_val,
        AIC = AIC(model),
        BIC = BIC(model)
    )
}

models_list <- list(
    "All Untransformed Variables" = model1,
    "Reducing Multicollinearity" = model2,
    "Full Model with Backward Selection" = model_full
)

# Apply the function to all three models, explicitly passing the training split
model_comparison <- imap_dfr(models_list, ~ {
    get_logistic_metrics(.x, .y, crime_test_split)
}) |>
    arrange(`Deviance`)

# Transpose the data and convert to a data frame
model_t <- as.data.frame(t(model_comparison))

# 3. Display the table
knitr::kable(
    model_t,
    digits = 4,
```

```
    caption = "Evaluation of Candidate Models on Training Dataset"
)

best_model_info <- model_comparison |> slice(1)

best_model_obj <- models_list[[best_model_info$Model]]
```

For the logistic models, we decided to use the deviance as our primary criterion on the training dataset, which was attained by the `best_model_info$Model`. We've chosen deviance as our primary metric because it directly measures the goodness of fit for logistic regression models, with lower deviance indicating a better fit to the training data. Given our model results, each model had a fairly similar accuracy with our `best_model_info$Model` having the highest accuracy of `round(best_model_info$Accuracy, 3)`. It also had the lowest error rate of `round(best_model_info$Error_Rate, 3)` and was comparable with the other top models in sensitivity and specificity. Overall, this model had the best balance of fit and predictive performance on the training data, which is why we selected it for further evaluation on the test set and for generating predictions on the evaluation set.

Given that our target variable was fairly balanced (49% of all observations had a target value of 1), and our accuracy, Precision, and Sensitivity show that we greatly outperform a random classifier, we are confident that our model is performing well.

```
# Display the summary of the selected Stepwise model to interpret coefficients and significanc
summary(best_model_obj)
```

To make inferences from our top performing model, we can look at the significance of the predictors. The logistic regression model identifies several key environmental and structural factors that significantly predict whether a suburb's crime rate will exceed the median. The most dominant predictor is the concentration of nitrogen oxides (nox), where higher levels are strongly associated with a higher probability of the area being high-crime ($z=4.670, p<0.001$). Additionally, the pupil-teacher ratio (ptratio) and the proportion of older homes (age) built before 1940 are both significant positive predictors, suggesting that areas with crowded schools and aging infrastructure are more likely to see elevated crime rates. The model also highlights that as the distance to employment centers (log_dis) increases, so does the likelihood of high crime, along with a significant effect from the ratio of rooms to lower-status population (rm_lstat_ratio). Interestingly, many traditional economic indicators, such as median home values (medv), property tax rates (log_tax), and zoning for large lots (log_zn), did not reach statistical significance, indicating that in this specific dataset, industrial and demographic markers are more reliable indicators of crime than general wealth or proximity to the Charles River.

7 Results

```
# Generate predictions on the evaluation dataset using the selected model
eval_predictions <- predict(best_model_obj, newdata = crime_eval_clean, type = "response")

crime_eval_results <- crime_eval_clean |>
    mutate(
        # If probability > 0.5, it's a 1 (high crime), otherwise 0
        PREDICTED_TARGET = ifelse(eval_predictions > 0.5, 1, 0)
    )

# Write predictions to a csv
write_csv(crime_eval_results, "crime_evaluation_predictions.csv")

# Display the first 10 predictions in the rendered PDF
knitr::kable(
    head(crime_eval_results |> select(PREDICTED_TARGET), 10),
    caption = "Excerpts of Stepwise Model Win Predictions for Evaluation Set"
)
```

Given our evaluation set and our stepwise model, we are able to create predictions on the evaluation set on which neighborhoods are classified as high-crime or low-crime. The first 10 predictions are shown in the table above. We write these predictions to a CSV file for submission.

8 Conclusion

In conclusion, our `best_model_info$Model` model, which was selected based on deviance and other performance metrics, demonstrates strong predictive capabilities for classifying neighborhoods as high-crime or low-crime based on a variety of environmental and structural factors. The significant predictors identified in the model provide valuable insights into the underlying drivers of crime in urban areas, highlighting the importance of air pollution, school crowding, aging infrastructure, and proximity to employment centers. These findings can inform policymakers and community leaders in their efforts to target interventions and allocate resources effectively to reduce crime rates. Future work could explore additional modeling techniques, such as regularization or ensemble methods, to further enhance predictive performance and interpretability.